Beyond Text: A Render-Aware, Semantically-Fused Defence Against Indirect Prompt Injection in Enterprise RAG Pipelines

1st Abu Nasar
Center for Cybersecurity Systems and Networks
Amrita Vishwa Vidyapeetham
Amritapuri, India
am.sc.p2csn25021@am.students.amrita.edu

2nd Niket Krishnan
Center for Cybersecurity Systems and Networks
Amrita Vishwa Vidyapeetham
Amritapuri, India
am.sc.p2csn25011@am.students.amrita.edu

3rd Devi Rajeev
Center for Cybersecurity Systems and Networks
Amrita Vishwa Vidyapeetham
Amritapuri, India
devir@am.amrita.edu

4th Vysakh Kani Kolil
Center for Cybersecurity Systems and Networks
Amrita Vishwa Vidyapeetham
Amritapuri, India
vysakh@am.amrita.edu

Abstract*—Formatting-based prompt-injection attacks hide malicious instructions inside PDFs, creating machine-extractable yet human-invisible threats that evade existing text-only defences. A render-aware hidden-text detector is introduced as the core contribution—the first component to identify adversarial prompt-injection instructions concealed through PDF formatting obfuscation across six technique families (zero-point fonts, white-on-white text, low-contrast rendering, and off-page placement), attack vectors fully machine-extractable yet invisible to human reviewers and unreachable by all existing text-based defences. This detector forms Layer 2 of a four-layer hybrid framework that additionally integrates VirusTotal file reputation (Layer 1), fine-tuned DistilBERT semantic classification (Layer 3), and an XGBoost fusion classifier producing a unified risk score in $[0.0, 1.0]$. An expanded hybrid benchmark of 11,050 samples is constructed spanning HackAPrompt text injections (10,000), synthetically obfuscated PDFs across six technique variants (1,000), and VirusTotal-verified malware (≈50). The system achieves macro $F_1 = 0.8770$ and ROC-AUC $= 0.9390$ (10-fold CV: $F_1 = 0.8845 \pm 0.0105$; bootstrap 95 % CI: [0.8627, 0.8918]). Adversarial robustness evaluation identifies a detection threshold between hidden-content scores of 0.10 and 0.20: single-technique obfuscation evades the framework, while two or more simultaneous render techniques are reliably detected. Multilingual evaluation across 514 verified non-English samples spanning 29 languages yields $F_1 = 0.9143$ (ROC-AUC $= 0.9672$). SHAP analysis confirms modality-specific complementarity: `semantic_score` dominates globally (mean $|\text{SHAP}| = 2.80$) while `hidden_score` delivers decisive signals exclusively for formatting-obfuscated samples. End-to-end latency is 837 ms, within the 2,000 ms target.

***Index Terms*—Indirect Prompt Injection, LLM Security, Render-Aware Detection, DistilBERT, XGBoost, Document Security, Fusion Classification, SHAP, RAG Security, Adversarial Machine Learning**

I. Introduction

The rapid integration of Large Language Models into enterprise document pipelines introduces security threats that traditional cybersecurity tools were not designed to address [1], [2]. **Indirect Prompt Injection (IPI)** embeds adversarial instructions in documents to manipulate LLM behaviour at inference time without any direct model access [3]. Organisations increasingly expose their pipelines to untrusted external documents through Retrieval-Augmented Generation (RAG) workflows [4], making the ingestion stage the primary exploitation surface [5], [6].

Existing defences are each blind to the other's threat class. File reputation systems detect known malware signatures but carry no semantic understanding [7]–[9]. Text-based classifiers and LLM guardrails analyse extracted content but have no visibility into the rendering properties of the source document [10]–[12]. This *modality gap* is actively exploited: adversaries embed injection instructions using zero-point fonts, white-on-white text, low-contrast rendering, or off-page placement—techniques invisible to human reviewers yet fully preserved during PDF text extraction [3], [13]. Attack success rates exceed 80 % on adversarial benchmarks [14], and the attack surface continues to expand through dialogue injection [11], optimisation-based attacks [15], prompt injection against LLM-integrated applications [16], [17], and adversarial prompt manipulation [18], [19].

The primary contribution is the **render-aware detector** (Layer 2)—a structural PDF analyser applying font-size, colour, contrast, and bounding-box metadata to identify formatting-concealed injections across six obfuscation-technique families, closing the modality gap that no text-based classifier can address. No prior work has applied structural document analysis specifically to prompt injection concealment. This detector is embedded within a four-layer hybrid framework whose contributions are: (1) a render-aware hidden-text detector via PyMuPDF metadata across six obfuscation families; (2) a four-layer framework combining VirusTotal [7], the render-aware detector, DistilBERT [20], and XGBoost fusion [21]; (3) an 11,050-sample benchmark

with 1,000 render-obfuscated PDFs covering six technique variants; (4) adversarial robustness evaluation characterising the detection threshold between scores 0.10 and 0.20; (5) multilingual evaluation across 29 languages ($F_1 = 0.9143$); and (6) SHAP interpretability with bootstrap CIs confirming modality-specific complementarity.

II. Related Work

Table I summarises the most relevant prior work. A consistent gap emerges: no existing system combines structural document analysis, file reputation, and semantic classification in a unified pre-ingestion framework. Text-based classifiers [10], [22] are blind to formatting-obfuscated injections; PDF structural detectors [13], [23], [24] target executable malware rather than semantic content; file reputation systems [7] provide no semantic understanding. Steganographic concealment further broadens the attack surface [19]. This work closes the gap by fusing all three detection modalities before text extraction.

TABLE I
Summary of Related Work

Ref.	Domain	Key Contribution	Gap vs. Ours
Greshake [3]	IPI Attacks	IPI taxonomy on real LLMs	No render-aware analysis
Schulhoff [14]	Benchmarks	HackAPrompt 600K+ prompts	No file-reputation or render layer
Yi [25]	Benchmarks	BIPIA across 25 LLMs	Blind to obfuscated docs
Shi [10]	Defences	CoT injection detection	Text-only; blind to obfuscation
Liu [22]	Defences	Formal attack/defence eval	No document structure
Maiorca [13]	PDF Malware	Structural feature survey	Targets malware, not injection
Biswas [23]	PDF Malware	ML/DL PDF comparison	No injection-intent analysis
Sanh [20]	Transformer	DistilBERT	Not applied to injection
Misquitta [7]	File Rep.	VirusTotal aggregation	No semantic layer

III. Methodology

A. Threat Model

The threat model assumes an attacker who can modify documents before LLM ingestion [1], [3], [26] but cannot access the model directly. Attack delivery occurs via: (1) poisoned documents in shared storage or email; (2) malware-embedded files; and (3) formatting-obfuscated injection text using white text, zero-point fonts, or off-page placement.

B. Dataset

A hybrid benchmark of 11,050 samples is constructed from three sources covering all three threat modalities. From HackAPrompt [14], 5,000 successful and 5,000 failed injection attempts are sampled; evaluation samples are drawn from rows 100,000+ to prevent overlap with DistilBERT fine-tuning data. Five hundred malicious PDFs are generated using ReportLab across six obfuscation techniques: (1) white text on white background; (2) zero-point font (0.1 pt); (3) off-page content positioning, detected via raw PDF content-stream parsing since `get_text()` silently clips out-of-viewport spans; (4) tiny white text (0.5 pt); (5) light-grey (`#F0F0F0`); and (6) low-contrast (`#E8E8E8`) text. Five hundred benign PDFs use normally visible text, together forming the 1,000-sample synthetic PDF subset. Approximately 50 confirmed malicious file hashes are queried via VirusTotal [7]. Split: $|\mathcal{D}_{\text{train}}| = 8{,}840$; $|\mathcal{D}_{\text{test}}| = 2{,}210$. Synthetic PDFs receive vt_score $= 0.0$ since freshly generated files are absent from VirusTotal's database—a limitation acknowledged in Section IV-H.

C. System Architecture

The framework processes every document through a sequential pipeline before it reaches the LLM (Fig. 1). Each layer produces a numerical risk score; all scores are concatenated into a nine-dimensional feature vector for XGBoost fusion.

1) Layer 1 — File Reputation (VirusTotal): Documents are SHA-256-hashed and queried against the VirusTotal API [7], aggregating 70+ antivirus engines: vt_score $=$ (malicious $+$ suspicious)/total_engines. Flagged files are immediately rejected. Mean latency: 768.2 ± 19.7 ms.

2) Layer 2 — Render-Aware Detection (PyMuPDF): PyMuPDF extracts per-span font size, colour, and bounding-box metadata. Four binary indicators capture the six obfuscation families: `has_tiny` ($\mathbf{1}[\min\{s_i\} < 1.0\,\text{pt}]$), `has_white` ($\mathbf{1}[\text{colour} \in \{\texttt{\#ffffff}, \texttt{\#fafafa}\}]$), `has_lc` ($\mathbf{1}[\text{colour} \in \{\texttt{\#f0f0f0}, \texttt{\#e8e8e8}\}]$), and `has_offp` ($\mathbf{1}[\text{content-stream coords} \notin \text{viewport}]$). The composite hidden-content score is:

$$\text{hidden_score} = \tfrac{1}{4}\big(\text{has_tiny} + \text{has_white} + \text{has_lc} + \text{has_offp}\big) \tag{1}$$

Mean latency: 32.3 ± 4.2 ms per document.

3) Layer 3 — Semantic Classification (DistilBERT): DistilBERT-base-uncased [20] is fine-tuned on 100,000 HackAPrompt samples [14] (AdamW, lr$= 2\times10^{-5}$, 3 epochs, batch 16, max 128 tokens). Standalone: $F_1 = 0.8257$, accuracy 82.58 %. Mean latency: 36.5 ± 53.4 ms.

4) Fusion Layer — XGBoost: Nine features $\mathbf{x} = [\text{sem}, \text{min_f}, \text{avg_f}, \text{tiny}, \text{white}, \text{lc}, \text{offp}, \text{hidden}, \text{vt}]$ are fused by XGBoost [21] ($n_{\text{est.}}$=200, depth=6, lr=0.1, subsample=0.8, θ=0.5). Inference overhead: 0.46 ± 0.37 ms.

IV. Results and Discussion

A. Baseline Comparison

Table II compares the fusion model against six baselines on the held-out test set ($n = 2{,}210$). Single-modality baselines (Render-Only, VT-Only) yield near-random F_1 as their signals activate for only small dataset subsets. The semantic-only DistilBERT baseline ($F_1 \approx 0.83$) is the strongest single-layer alternative but is blind to all formatting-obfuscated samples. XGBoost 10-fold cross-validation confirms statistical stability ($\sigma_{F_1} = 0.0105$), ruling out a favourable single-split artefact.

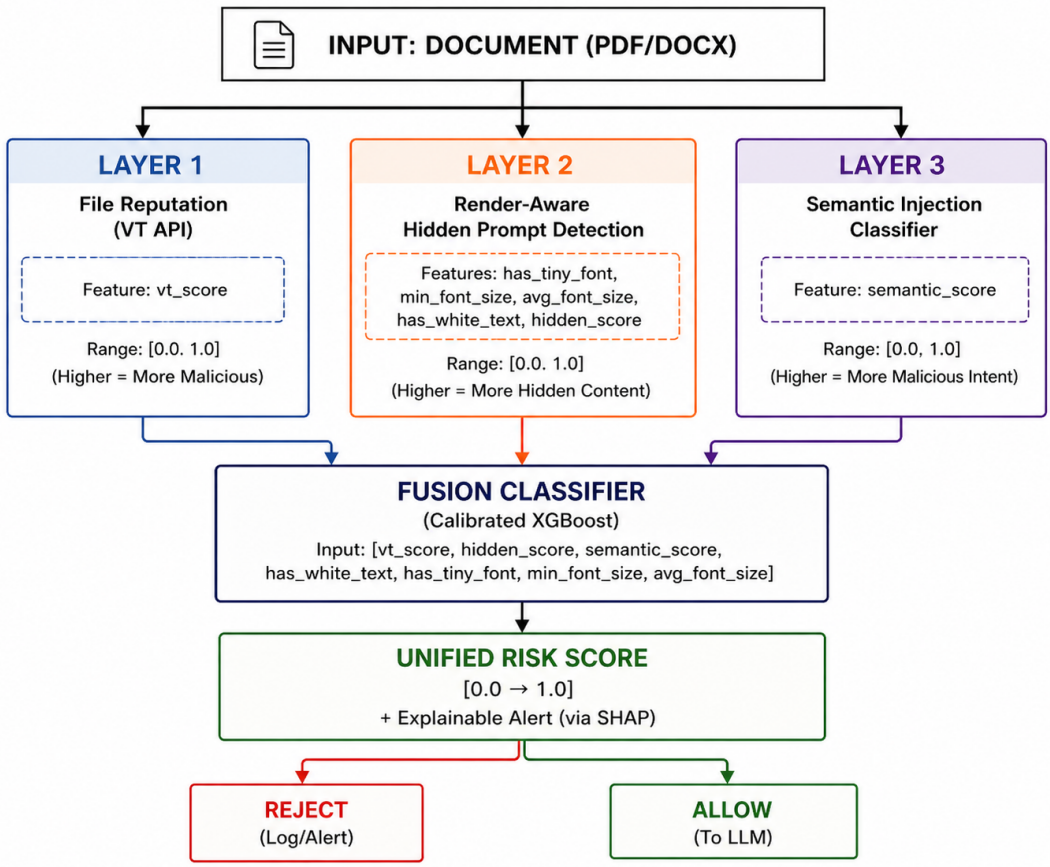


Fig. 1. Hybrid Document-Ingestion Security Framework. Each layer independently scores the document; XGBoost fuses scores into a unified risk output.

TABLE II
BASELINE COMPARISON ($n = 2{,}210$)

Method	F_1 (Macro)	Accuracy
Keyword Rules	0.60	0.60
Semantic Only (DistilBERT)	0.8257	0.8258
Render-Aware Only	0.3879	0.5240
VirusTotal Only	0.3363	0.4995
Logistic Regression	0.8750	0.8756
Random Forest	0.8483	0.8484
XGBoost Fusion (Ours)	**0.8770**	**0.8774**
XGBoost 10-Fold CV	$0.8845_{\pm 0.0105}$	$0.8848_{\pm 0.0104}$

B. Fusion Model Performance

Table III presents the full classification report. Bootstrap CIs (1,000 iterations) confirm statistical reliability: F_1 95 % CI = [0.8627, 0.8918]; AUC 95 % CI = [0.9287, 0.9494] (ROC-AUC = 0.9390). The confusion matrix (TN = 912, FP = 188, FN = 83, TP = 1,027) reveals a slight recall asymmetry favouring malicious detection (malicious recall: 0.93 vs. benign: 0.83).

C. Modality-Specific Error Analysis

All 271 prediction errors originate exclusively from the HackAPrompt modality; the synthetic PDF and VirusTotal subsets achieve zero errors ($F_1 = 1.000$ on both). Table IV

TABLE III
XGBOOST FUSION CLASSIFICATION REPORT ($n = 2{,}210$)

Class	Prec.	Rec.	F_1	Support
Benign	0.92	0.83	0.87	1,100
Malicious	0.85	0.93	0.88	1,110
Macro Avg	0.88	0.88	0.88	2,210
Accuracy			0.8774	
ROC-AUC			0.9390	[0.9287, 0.9494]

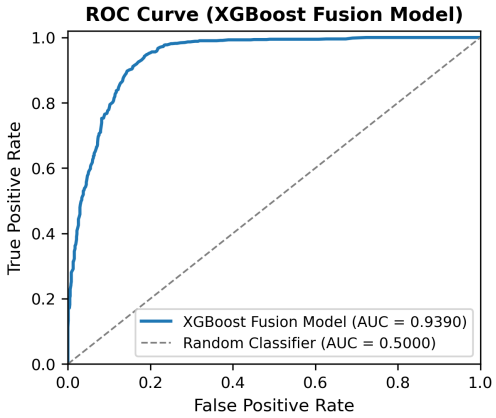


Fig. 2. ROC curve (AUC = 0.9390), demonstrating strong discrimination across all decision thresholds.

compares XGBoost and Logistic Regression per modality. Although the global F_1 gap is modest, XGBoost achieves decisive improvement on VirusTotal samples ($F_1 = 1.000$ vs. 0.7692, $n = 8$), directly demonstrating XGBoost's advantage on the minority-class samples where the linear model underperforms. Logistic regression's linear decision boundary misclassifies three of the eight VirusTotal-malicious test samples as benign, while XGBoost's tree-based splits classify all eight correctly.

TABLE IV
PER-MODALITY PERFORMANCE: XGBOOST VS. LOGISTIC REGRESSION

Modality	**XGBoost** F_1	**LR** F_1
HackAPrompt (n=2,004)	0.8645	0.8637
Synthetic PDF (n=198)	1.0000	1.0000
VirusTotal (n=8)*	1.0000	0.7692

*Single-class subset; F_1 for class 1 only.

D. Feature Importance and SHAP Analysis

Table V presents XGBoost gain and SHAP values side by side; Figs. 3 and 4 visualise the gain-based bar chart and SHAP beeswarm respectively. XGBoost gain ranks `has_low_contrast` first (0.3908) because it cleanly partitions the low-contrast PDF subset with near-perfect splits, yielding high information gain despite affecting only a fraction of samples. SHAP identifies `semantic_score` as globally dominant (mean $|\text{SHAP}| = 2.7952$) since it drives predictions for all 10,000 HackAPrompt samples. This divergence between gain and SHAP rankings—more pronounced here than in prior work due to the larger modality imbalance (10,000 HackAPrompt vs. 1,050 PDF + VT)—demonstrates why gain-based importance alone misleads architectural decisions in multi-modal imbalanced datasets [18], [19]. `hidden_score` delivers SHAP values of $+5$ to $+6$ exclusively for render-obfuscated inputs; `vt_score` fires only for confirmed malware. Researchers applying XGBoost to security tasks with modality-imbalanced datasets should cross-validate both metrics before drawing architectural conclusions.

TABLE V
FEATURE IMPORTANCE: XGBOOST GAIN VS. MEAN |SHAP|

Feature	**Layer**	**XGB Gain**	**Mean** $\lvert\text{SHAP}\rvert$
has_low_contrast	Render-Aware	0.3908	0.0855
hidden_score	Render-Aware	0.2045	0.4195
has_offpage	Render-Aware	0.1649	0.0206
has_tiny_font	Render-Aware	0.1140	0.0259
vt_score	VirusTotal	0.0542	0.0507
has_white_text	Render-Aware	0.0362	0.0033
min_font_size	Render-Aware	0.0190	0.1047
semantic_score	Semantic	0.0163	2.7952
avg_font_size	Render-Aware	0.0002	0.0120

E. Adversarial Robustness

Adversarial PDFs are constructed with surface text designed to yield semantic_score ≈ 0.0075, successfully evading

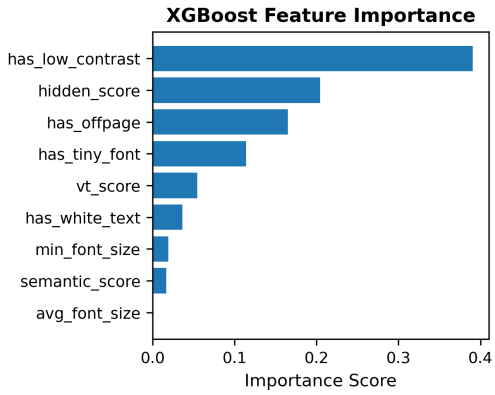


Fig. 3. XGBoost gain-based feature importances. `has_low_contrast` ranks highest by split gain due to clean partitioning of the low-contrast PDF subset.

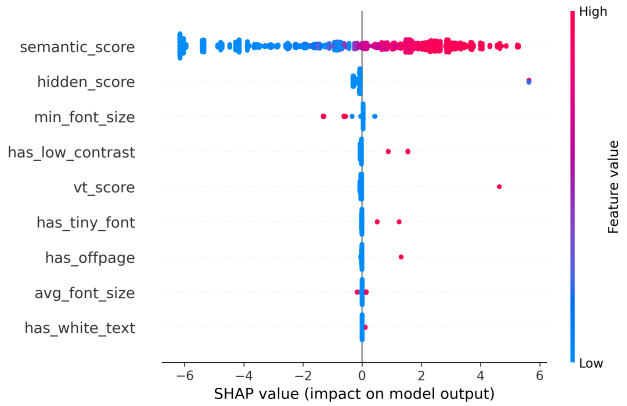


Fig. 4. SHAP beeswarm plot. `semantic_score` dominates (mean $|\text{SHAP}| = 2.80$); `hidden_score` delivers large positive values ($\approx +5$ to $+6$) exclusively for render-obfuscated samples; `vt_score` fires only for confirmed malware.

Layer 3, while `hidden_score` is varied by controlling the number of simultaneous obfuscation techniques. Table VI reports the detection rate per level on the retrained fusion model. A clear threshold exists between scores 0.10 and 0.20: single-technique obfuscation evades the framework, but two or more simultaneous render techniques are reliably detected—requiring attackers to apply at least two formatting techniques simultaneously. This constitutes a *multi-layer adversarial scenario*: Layer 3 (semantic) is actively evaded by the carefully crafted surface text, while Layer 2 (render-aware) detection is stressed across varying technique counts. The results confirm defence-in-depth: successful semantic-layer evasion alone is insufficient when render-layer signals are present. Conversely, minimal render obfuscation (hidden_score $= 0.10$) does not produce a sufficiently strong Layer 2 signal to compensate for the absent semantic signal, yielding zero detections. The evasion boundary therefore lies at the simultaneous activation of at least two render indicators.

F. Multilingual Evaluation

To assess cross-lingual generalisation, 5,000 non-ASCII HackAPrompt samples are language-verified using

TABLE VI
ADVERSARIAL ROBUSTNESS: DETECTION RATE BY EVASION LEVEL ($n = 10$ PER LEVEL)

hidden_score	Techniques Active	Detected
0.10	One (minimal)	0/10 (0 %)
0.20	Two (+ low-contrast)	10/10 (100 %)
0.35	Three (+ tiny font)	10/10 (100 %)
0.50	Three	10/10 (100 %)
0.75	Four	10/10 (100 %)
1.00	Maximum	10/10 (100 %)

`langdetect`; 2,733 true non-English samples spanning 29 languages are identified and balanced to 514 samples (257 per class). Table VII reports the results. The aggregate non-English $F_1 = 0.9143$ (ROC-AUC = 0.9672) substantially exceeds the semantic-only baseline observed on the full mixed test set ($F_1 = 0.8257$, Table II), indicating the DistilBERT semantic layer generalises well to non-English injection text despite English-only fine-tuning. Strongest per-language results include Bulgarian (bg: $F_1 = 0.9417$, $n = 25$), Simplified Chinese (zh-cn: 0.9411, $n = 88$), and Italian (it: 0.9412, $n = 17$). Weaker results are observed for Dutch (nl: 0.7979), Spanish (es: 0.8101), and Korean (ko: 0.8599). Welsh (cy: $F_1 = 0.4762$, $n = 11$) is an outlier due to small sample size. Cross-lingual performance reflects shared structural injection patterns across language boundaries; production deployment requires multilingual fine-tuning (XLM-R or mBERT).

TABLE VII
MULTILINGUAL EVALUATION ($n = 514$, 29 LANGUAGES)

Class	Prec.	Rec.	F_1	Support
Benign	0.95	0.88	0.91	257
Malicious	0.88	0.95	0.92	257
Macro Avg	0.92	0.91	0.91	514
ROC-AUC			0.9672	

G. Latency Analysis and Production Considerations

Table VIII and Fig. 5 report the per-component latency. Total end-to-end latency of 837 ms meets the 2,000 ms operational target. VirusTotal dominates at 768 ms (91.7 %); local computation (Layers 2 and 3) adds only 68.7 ms. Two production mitigations are identified: (1) **asynchronous parallel execution** reduces effective latency to $\max(768, 32, 36) \approx 768$ ms; and (2) **SHA-256 hash caching** reduces repeated-document overhead to ≈69 ms [4].

H. Discussion

The central finding is *modality-specific complementarity*: each layer fires decisively within its own threat class and contributes negligibly outside it, confirmed by SHAP analysis (Table V), per-modality breakdown (Table IV), and adversarial robustness (Table VI). The gain/SHAP divergence is more pronounced in the expanded dataset due to the larger modality imbalance (10,000 HackAPrompt vs. 1,050 PDF + VT samples), reinforcing the necessity of cross-validating both metrics before drawing architectural conclusions.

TABLE VIII
END-TO-END LATENCY PER DOCUMENT

Component	Mean (ms)	Std (ms)
Layer 1 — VirusTotal	768.22	19.72
Layer 2 — Render-Aware	32.27	4.21
Layer 3 — DistilBERT	36.46	53.35
Fusion — XGBoost	0.46	0.37
Total End-to-End	**837.41**	—
Target (< 2,000 ms)	✓ **MET**	

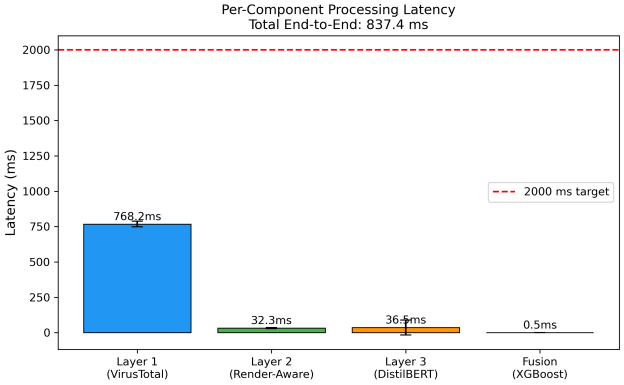


Fig. 5. Per-component processing latency. VirusTotal accounts for 91.7 % of total latency; local computation (Layers 2 and 3) contributes only 68.7 ms. Dashed line marks the 2,000 ms target.

Several limitations are acknowledged. Synthetic PDFs receive vt_score = 0.0 since freshly generated files are absent from VirusTotal's database; this reflects a real-world constraint as novel malware hashes require time to propagate to reputation databases. The render-aware detector's fixed colour indicators cover six technique variants but would miss novel near-white colours outside the training distribution; a perceptual luminance threshold ($L = 0.299R + 0.587G + 0.114B > 0.85$; ITU-R BT.601) would provide broader coverage without enumeration. The adversarial threshold finding (Section IV-E) confirms that single-technique obfuscation can evade the framework, motivating larger adversarial training sets with increased technique diversity. The multilingual evaluation (Section IV-F), while promising, covers only 514 samples across 29 languages; production deployment in non-English enterprise settings requires dedicated per-language fine-tuning. The VirusTotal test subset is also small and single-class ($n = 8$), so its reported F_1 reflects class-1 performance only and warrants a larger, balanced malware sample before drawing strong conclusions about Layer 1's standalone reliability. Validation on real-world enterprise documents beyond synthetic PDFs remains the highest-priority direction for future work.

V. CONCLUSION

A four-layer hybrid framework for pre-ingestion detection of indirect prompt injection has been presented, with the render-aware detector as the headline contribution closing the

modality gap left by all existing text-based and reputation-based defences [1], [3]. On an 11,050-sample benchmark spanning three threat modalities, the system achieves a held-out macro $F_1 = 0.8770$ (10-fold CV: $F_1 = 0.8845 \pm 0.0105$; CV ROC-AUC 0.9455 ± 0.0080; bootstrap 95 % CI: $[0.8627, 0.8918]$) within 837 ms end-to-end latency. Adversarial robustness evaluation establishes a detection threshold between hidden-content scores of 0.10 and 0.20: single-technique obfuscation evades the framework while multi-technique attacks are reliably caught. Multilingual evaluation across 514 non-English samples spanning 29 languages yields $F_1 = 0.9143$, confirming structural generalisation with per-language caveats. SHAP analysis confirms that each layer dominates exclusively within its own threat modality—a complementarity that no single-layer approach can replicate. Future work will expand the synthetic PDF and VirusTotal datasets, adopt luminance-based colour thresholding, evaluate multilingual base models (XLM-R, mBERT), implement asynchronous parallel execution, and validate the framework on real-world enterprise document corpora.

REFERENCES

[1] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From chatgpt to threatgpt: Impact of generative ai in cybersecurity and privacy," *IEEE Access*, vol. 11, pp. 80 218–80 245, 2023.

[2] K. Achuthan, S. Ramanathan, and R. Raman, "Securing the metaverse: Machine learning–based perspectives on risk, trust, and governance," *International Journal of Information Management Data Insights*, vol. 5, no. 2, p. 100356, 2025.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[4] R. Ruparel, A. Jatavallabha, and S. Maringanti, "Prompting for llm security and rag: A survey from zero-shot to automatic prompt optimization (apo) and prompt-injection defenses," in *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*. IEEE, 2026, pp. 1–6.

[5] K. Achuthan, S. Ramanathan, S. Srinivas, and R. Raman, "Advancing cybersecurity and privacy with artificial intelligence: current trends and future research directions," *Frontiers in big data*, vol. 7, p. 1497535, 2024.

[6] N. Devnath, B. H. Kumar, A. Sasikumar, and D. Rajeev, "Enhancing adversarial malware generation with gans and large language models," in *2026 International Conference on Computing, Communication, Control and Cyber-Physical Systems (I5CPS)*. IEEE, 2026, pp. 1–5.

[7] J. Misquitta and A. K, "A comparative study of malicious url detection: Regular expression analysis, machine learning, and virustotal api," *Research Square*, Nov. 2023, preprint. [Online]. Available: https://doi.org/10.21203/rs.3.rs-3685949/v1

[8] A. K. Mathew, M. Ashwin, V. K. Kolil, and D. Rajeev, "Counterfactual explanations for enhanced interpretability in cross-site scripting (xss) detection," in *2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA)*. IEEE, 2025, pp. 781–790.

[9] K. Achuthan, S. Sankaran, S. Roy, and R. Raman, "Integrating sustainability into cybersecurity: insights from machine learning based topic modeling," *Discover Sustainability*, vol. 6, no. 1, p. 44, 2025.

[10] L. Shi, Y. Kang, J. Hu, X. Li, and M. Yang, "Meticulous thought defender: Fine-grained chain-of-thought (cot) for detecting prompt injection attacks of large language models," *IEEE Access*, 2025.

[11] W. Meng, F. Zhang, W. Yao, Z. Guo, Y. Li, C. Wei, and W. Chen, "Dialogue injection attack: Jailbreaking llms through context manipulation," *IEEE Transactions on Information Forensics and Security*, 2026.

[12] K. Achuthan and S. Khobragade, "Cyberattacks, perception, and market impact: Financial and social media dynamics of breach disclosure," *Computers & Security*, p. 104862, 2026.

[13] D. Maiorca, B. Biggio, and G. Giacinto, "Towards adversarial malware detection: Lessons learned from pdf-based attacks," *ACM Computing Surveys*, vol. 52, no. 4, p. 1–36, Aug. 2019. [Online]. Available: http://dx.doi.org/10.1145/3332184

[14] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Anati, V. Tagliabue, A. L. Kost, C. Carnahan, and J. Boyd-Graber, "Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global scale prompt hacking competition," 2024. [Online]. Available: https://arxiv.org/abs/2311.16119

[15] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to llm-as-a-judge," 2025. [Online]. Available: https://arxiv.org/abs/2403.17710

[16] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt injection attack against llm-integrated applications," 2025. [Online]. Available: https://arxiv.org/abs/2306.05499

[17] K. Kandasamy, S. Srinivas, K. Achuthan, and V. P. Rangan, "Digital healthcare-cyberattacks in asian organizations: an analysis of vulnerabilities, risks, nist perspectives, and recommendations," *IEEE access*, vol. 10, pp. 12 345–12 364, 2022.

[18] X. Xu, K. Kong, N. Liu, L. Cui, D. Wang, J. Zhang, and M. Kankanhalli, "An llm can fool itself: A prompt-based adversarial attack," *arXiv preprint arXiv:2310.13345*, 2023.

[19] N. Wang, K. Walter, Y. Gao, and A. Abuadbba, "Understanding the adversarial landscape of large language models through the lens of attack objectives," *IEEE Security & Privacy*, vol. 24, no. 1, pp. 53–60, 2026.

[20] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," 2020. [Online]. Available: https://arxiv.org/abs/1910.01108

[21] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. ACM, Aug. 2016, p. 785–794. [Online]. Available: http://dx.doi.org/10.1145/2939672.2939785

[22] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[23] S. Biswas and A. H. Mohammad Nibras, "Detecting malware in pdf files: A comparative analysis of machine learning and deep learning approaches," in *2025 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, 2025, pp. 1–5.

[24] S. Chandran, S. R. Syam, S. Sankaran, T. Pandey, and K. Achuthan, "From static to ai-driven detection: A comprehensive review of obfuscated malware techniques," *IEEE Access*, 2025.

[25] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD '25. ACM, Jul. 2025, p. 1809–1820. [Online]. Available: http://dx.doi.org/10.1145/3690624.3709179

[26] S. S. Kumar, M. Cummings, and A. Stimpson, "Strengthening llm trust boundaries: A survey of prompt injection attacks," in *2024 IEEE 4th International Conference on Human-Machine Systems (ICHMS)*, 2024, pp. 1–6.